```sql
create database companyqueries;
use companyqueries;
create table Departments(dept_id int primary key , dept_name varchar(50));
INSERT INTO Departments (dept_id, dept_name) VALUES(1, 'HR'),(2, 'IT'),(3, 'Finance'),(4, 'Marketing'),(5, 'Sales');
create table Employees(emp_id int primary key ,emp_name varchar(50), dept_id int,foreign key (dept_id) references Departments(dept_id), salary decimal(20,2), manager_id int null, foreign key(manager_id)
references Employees(emp_id), join_date date);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id, join_date) VALUES
(101, 'Alice', 1, 60000, NULL, '2020-01-15'),
(102, 'Bob', 1, 45000, 101, '2021-03-10'),
(103, 'Carol', 1, 45000, 101, '2022-06-20'),
(201, 'David', 2, 90000, NULL, '2019-02-01'),
(202, 'Eve', 2, 70000, 201, '2020-07-15'),
(203, 'Frank', 2, 70000, 201, '2021-09-12'),
(204, 'Grace', 2, 60000, 201, '2023-01-05'),
(301, 'Henry', 3, 80000, NULL, '2018-04-26'),
(302, 'Ivy', 3, 50000, 301, '2020-10-10'),
(303, 'Jack', 3, 40000, 301, '2022-02-25'),
(401, 'Kate', 5, 55000, NULL, '2021-05-18'),
(402, 'Leo', 5, 45000, 401, '2023-08-01');
create table  Projects(project_id int primary key, project_name varchar(50), dept_id int,foreign key (dept_id)references Departments(dept_id), budget decimal(10,2));
INSERT INTO Projects (project_id, project_name, dept_id, budget) VALUES
(1001, 'ERP System', 2, 200000),
(1002, 'Website', 2, 120000),
(1003, 'Financial Audit', 3, 80000),
(1004, 'Recruitment', 1, 50000);
create table EmployeeProjects(emp_id int,project_id int, hours_worked int, primary key(emp_id,project_id),foreign key (emp_id)references Employees(emp_id),foreign key (project_id)references Projects(project_id));
INSERT INTO EmployeeProjects (emp_id, project_id, hours_worked) VALUES
(201, 1001, 50),
(202, 1001, 40),
(203, 1001, 40),
(204, 1001, 30),

(204, 1002, 40),
(203, 1002, 50),
(202, 1002, 30),
(201, 1002, 20),

(301, 1003, 30),
(302, 1003, 30),
(303, 1003, 20),

(102, 1004, 20),
(103, 1004, 20),
(101, 1004, 10);
create table Orders(order_id int primary key, customer_id int, order_date date, amount decimal(10,2), status varchar(10));
INSERT INTO Orders (order_id, customer_id, order_date, amount, status) VALUES
(10001, 1, '2026-01-10', 40000, 'Completed'),
(10002, 1, '2026-02-15', 30000, 'Completed'),
(10003, 1, '2026-03-20', 50000, 'Completed'),
(10004, 2, '2026-02-05', 20000, 'Completed'),
(10005, 2, '2026-03-12', 15000, 'Completed'),
(10006, 2, '2026-04-18', 25000, 'Completed'),
(10007, 3, '2026-01-22', 10000, 'Completed'),
(10008, 3, '2026-03-05', 15000, 'Completed'),
(10009, 4, '2026-04-10', 15000, 'Completed'),
(10010, 5, '2026-01-08', 30000, 'Completed'),
(10011, 5, '2026-02-14', 30000, 'Completed'),
(10012, 5, '2026-03-25', 40000, 'Completed'),
(10013, 6, '2026-02-10', 20000, 'Pending'),
(10014, 6, '2026-03-15', 10000, 'Cancelled'),
(10015, 3, '2026-04-01', 50000, 'Cancelled');

select * from Departments;
select * from Employees;
select * from EmployeeProjects;
select * from Projects;
select * from Orders;

-- 1.Show department name, employee count, average, minimum and maximum salary. Include departments with no employees.
select d.dept_name ,count(e.emp_id) as emp_count,avg(e.salary),min(e.salary),max(e.salary) from Departments d left join Employees e on d.dept_id=e.dept_id group by d.dept_name ;

-- 2.Using a window function, return employee(s) earning the second distinct highest salary in each department.
with ranked as(select emp_name,dept_id,salary ,dense_rank() over(partition by dept_id order by salary desc) as salary_rank from Employees) select emp_name, dept_id,salary from ranked where salary_rank=2;

-- 3.Find employees earning more than their own department average without hard-coding departments.
select e.emp_name,e.emp_id from Employees e where salary >(select avg(e2.salary) from Employees e2 where e2.dept_id=e.dept_id);
-- OR USE WINDOW FUNCTION
select emp_name,emp_id from(select emp_name,emp_id,salary,avg(salary) over(partition by dept_id) as dept_avg from Employees) e where salary>dept_avg;

-- 4.For each manager, show direct-report count, total team salary and manager salary; keep managers with at least two reports.
select m.emp_id,m.emp_name as manager_name, count(e.emp_id) as direct_count, sum(e.salary) as team_salary , m.salary as manager_salary from Employees m join Employees e on e.manager_id=m.emp_id group by m.emp_id,m.emp

-- 5.For each project, rank employees by total hours and return the top two contributors, handling ties correctly.
with hours as (select project_id ,emp_id, sum(hours_worked) as total_hours from EmployeeProjects group by project_id,emp_id),
ranking as (select project_id, emp_id, total_hours, dense_rank() over(partition by project_id order by total_hours desc)as rankHour from hours)
select project_id,emp_id,total_hours ,rankHour from ranking where rankHour<=2;

-- 6.Find departments that contain employees but have no matching project.
select d.dept_id,d.dept_name from Departments d where exists(select 1 from Employees e where e.dept_id =d.dept_id) and not exists (select 1 from Projects p where p.dept_id=d.dept_id);

-- 4.For each manager, show direct-report count, total team salary and manager salary; keep managers with at least two reports.
select m.emp_id,m.emp_name as manager_name, count(e.emp_id) as direct_count, sum(e.salary) as team_salary , m.salary as manager_salary from Employees m join Employees e on e.manager_id=m.emp_id group by
m.emp_id,m.emp_name,m.salary having count(e.emp_id)>=2;

-- 5.For each project, rank employees by total hours and return the top two contributors, handling ties correctly.
with hours as (select project_id ,emp_id, sum(hours_worked) as total_hours from EmployeeProjects group by project_id,emp_id),
ranking as (select project_id, emp_id, total_hours, dense_rank() over(partition by project_id order by total_hours desc)as rankHour from hours)
select project_id,emp_id,total_hours ,rankHour from ranking where rankHour<=2;

-- 6.Find departments that contain employees but have no matching project.
select d.dept_id,d.dept_name from Departments d where exists(select 1 from Employees e where e.dept_id =d.dept_id) and not exists (select 1 from Projects p where p.dept_id=d.dept_id);

-- 7.For completed orders, show order date, daily revenue and cumulative revenue ordered by date.
with dailyRevenue as (select order_date,sum(amount)as daily_revenue from Orders where status='Completed' group by order_date) select order_date ,daily_revenue, sum(daily_revenue) over(order by order_date)
as cumulative_revenue from dailyRevenue order by order_date;

-- 8.Classify each customer by completed-order value: PLATINUM >=100000, GOLD >=50000, SILVER >=20000, else BRONZE.
with order_value as (select customer_id ,sum(amount) as total_value from Orders where status='Completed' group by customer_id)select customer_id,total_value,
case when total_value>=100000 then 'PLATINUM' when total_value>=50000 then 'GOLD' when total_value>=20000 then 'SILVER' else 'BRONZE' end as customer_classification from order_value;

-- 9.Identify customers with at least one order in three consecutive calendar months using CTEs/window functions.
with months as (select distinct customer_id,date_format(order_date,'%Y-%m-01'))

-- 10.For each department, return highest-paid and lowest-paid employee(s) and the salary difference; preserve ties.
select dept_id, emp_id, emp_name, salary,
case when salary = highest_salary then 'highest'
when salary = lowest_salary then 'lowest' end as salary_type, highest_salary - lowest_salary as salary_difference
from (select dept_id,emp_id,emp_name,salary,max(salary) over (partition by dept_id) as highest_salary,min(salary) over (partition by dept_id) as lowest_salary from employees) e where salary = highest_salary or
salary = lowest_salary order by dept_id, salary desc;
```